

Conclusion to Building APIs

 [. \(https://github.com/learn-co-curriculum/python-p4-conclusion-to-building-apis\)](https://github.com/learn-co-curriculum/python-p4-conclusion-to-building-apis)  [. \(https://github.com/learn-co-curriculum/python-p4-conclusion-to-building-apis/issues/new\)](https://github.com/learn-co-curriculum/python-p4-conclusion-to-building-apis/issues/new)

Learning Goals

- Build APIs to handle GET, POST, PATCH, and DELETE requests.

Key Vocab

- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.

Conclusion

You've learned a lot in this module:

- Serializing SQLAlchemy records into JSON with SQLAlchemy-serializer and jsonify.
- Defining routes that only accept specific methods.
- Displaying database records for the client in a machine-readable format.

- Creating database records with POST request forms.
- Updating database records with PATCH request forms.
- Deleting database records with DELETE requests.

These are the fundamentals of API development. There are more HTTP request methods out there and some techniques for improving on your API design, but you're now ready to connect your databases to the internet and modify them with the data you get back.

In the next module, we'll discuss Representational State Transfer (REST) protocol and how it informs good API design.

Resources

- [Flask - Pallets](https://flask.palletsprojects.com/en/2.2.x/) ➞ (https://flask.palletsprojects.com/en/2.2.x/)
- [HTTP request methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)
- [What is an API? - MuleSoft](https://www.mulesoft.com/resources/api/what-is-an-api) ➞ (https://www.mulesoft.com/resources/api/what-is-an-api)
- [GET - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET)
- [POST - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST)
- [PATCH - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/PATCH) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/PATCH)
- [DELETE - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/DELETE) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/DELETE)
- [flask.json jsonify Example Code - Full Stack Python](https://www.fullstackpython.com/flask-json-serialize-examples.html) ➞ (https://www.fullstackpython.com/flask-json-serialize-examples.html)
- [SQLAlchemy-serializer - PyPI](https://pypi.org/project/SQLAlchemy-serializer/) ➞ (https://pypi.org/project/SQLAlchemy-serializer/)